# 학습 노트 (4). 내 칩의 견적과 다섯 베릴로그의 계보

오픈소스 반도체 설계 및 제작 종합설계 (2026 Fall)
공과대학 기계공학부 안준영 학번 2024-11078 2026년 10월 16일

## 들어가며

이번 주 과제는 크게 세 갈래였다. Q.13과 Q.14의 Talks 디렉토리에서 인공지능과 주고받은 대화를 검토하는 일, simulation 디렉토리에서 make를 실행해 서로 다른 베릴로그 RTL을 링크하고 그 차이를 이해하는 일, 그리고 검증 보드 배포를 앞두고 개발 도구를 설치하고 Makefile이 각 단계에서 어떤 도구를 부르는지 살펴보는 일이다. Q.13과 Q.14의 내용이 갱신되었다고 해서 저장소를 다시 내려받고 시작했다.

## 1. 인공지능과의 대화: 내 칩의 견적 내기

Q.14의 Talks 디렉토리에는 여섯 개의 문답(talks_1.txt에서 talks_3.txt)과 세 개의 베릴로그 파일이 들어 있다. 문답은 결국 질문 하나에 대한 것이다. 바이브 코딩으로 만든 게임 RTL을 ETRI 0.5um 공정의 내 칩 제작 서비스로 만들 수 있는가 하는 견적이다.

### 1.1 면적 폭발의 두 원인

인공지능은 먼저 MPW 방식의 단위 칩 면적이 1.5mm x 1.5mm로 정해져 있다는 점을 짚고, 원본 RTL에서 면적을 키우는 요인 두 가지를 가리켰다. 첫째는 스무다섯 개 인베이더의 충돌을 검사하는 중첩 for 문이다. 베릴로그의 for 문은 시간의 반복이 아니라 회로의 복제다. 조합 논리 블록에 쓰인 이중 루프는 합성 과정에서 스무다섯 벌의 검사 회로로 펼쳐진다. 둘째는 c x 12 같은 곱셈이다. 0.5um 공정에서 곱셈기 하나가 수백 게이트를 차지하는데, 루프와 겹쳐 수십 개가 만들어진다. 대화에서 제시된 추정으로 원본은 약 12,000 게이트를 넘고, 배선 여유까지 고려한 이 공정의 한계가 만 게이트 안팎이라 초과 위험이 컸다.

### 1.2 순차 탐색 구조로의 재설계

해법은 회로의 구조를 바꾸는 것이었다. chk_idx라는 다섯 비트 카운터를 두고 매 클럭에 인베이더 한 개씩만 검사하는 순차 탐색으로 바꾸면 검사 회로 한 벌이면 충분하다. 곱셈은 시프트와 덧셈으로 치환한다. c x 12는 (c 〈〈 3) + (c 〈〈 2)와 같고, r x 8은 (r 〈〈 3)이다. 스캔 한 프레임에 비해 스물다섯 클럭의 순차 탐색이 훨씬 빠르게 끝나므로 게임 동작에는 영향이 없다. 이 재설계가 space_invaders_engine_glcd.v_1이다. 인공지능의 추정으로 게이트 수는 2,000 안팎으로 줄어 원본 대비 약 80퍼센트의 면적을 덜어냈다.

### 1.3 사람이 잡아낸 두 개의 버그

이 대화에서 가장 인상적인 부분은 그 다음이다. 인공지능이 내놓은 v_1에는 c와 t_invader_x, t_invader_y의 선언이 빠져 있었다. 사람의 지적에 선언을 보완한 것이 v_2다. 그런데 v_2에는 더 성가신 결함이 남아 있었다. 인베이더 미사일의 발사 행을 찾는 for 문의 루프 변수 loop_r이 3비트 부호 없는 레지스터로 선언되어 있어, 0에서 1을 빼는 순간 7로 언더플로가 일어나고 종료 조건 loop_r 〉= 0이 영원히 참이 된다. 사람이 다시 지적했고, loop_r을 integer로 바꾼 v_3가 최종본이 되었다.

이 대화를 읽으며 지난주의 생각이 실제 사례를 얻었다고 느꼈다. 인공지능은 면적 최적화라는 큰 그림에서는 능숙했다. 그러나 선언이 빠진 것이나 부호 없는 변수의 언더플로처럼 기본적이지만

치명적인 결함은 사람이 코드를 읽어야 걸러졌다. Fig. 1은 대화의 마지막 부분으로, 재설계된 코드의 면적 평가가 통과되었다는 인공지능의 종합 의견이다.

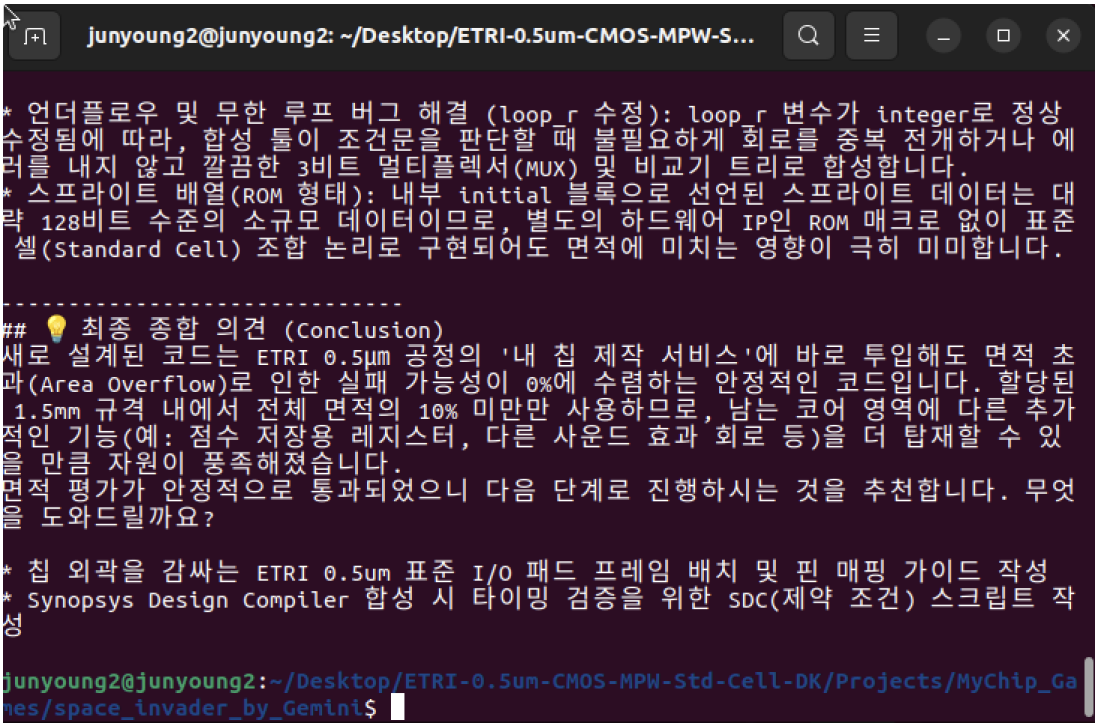


Fig. 1. Talks 대화의 마지막 부분. 재설계된 코드의 면적 평가를 통과했다는 인공지능의 종합 의견.

## 2. 다섯 베릴로그의 계보

Q.13과 Q.14의 simulation 디렉토리에서 make를 실행하면 여러 버전의 RTL 중 하나를 골라 링크한다. 두 디렉토리의 후보 목록이 서로 다른 이야기를 담고 있어서, diff로 버전 사이의 차이를 추적했다.

### 2.1 Q.13의 다섯 버전: 시뮬레이션에서 실물을 향해

Q.13의 후보는 0번 Org_wild_cannon부터 4번 Ok까지다. 계보를 정리하면 다음과 같다.

- 0번은 지난주 Q.11의 원본이다. 발사 버튼을 레벨로 읽어 꾹 누르면 포탄이 연사되고, 키 입력이 매 클럭 반영되어 포가 비정상적으로 빨리 움직인다. 이름의 wild cannon이 증상 그대로다.
- 1번은 상태를 원핫 인코딩으로 바꾸고 STATE_CHK_BTN을 추가했다. btn_fire_dly 레지스터로 버튼의 상승 에지를 검출해 단발 사격이 되었고, 화면 스캔의 끝(scan_x가 127이고 scan_y가 63)에서 chk_btn을 세워 좌우 이동이 프레임당 한 번만 일어나게 했다. 지난주 키보드 이상 반응의 정답이 이 구조다. Fig. 2에 이 차이를 보여 주는 diff의 일부가 있다.
- 2번과 3번은 에지 검출과 player_hit 플래그를 하나씩 떼어 본 실험 버전이다. 원인을 분리하려고 기능을 하나씩 빼 본 흔적으로 읽힌다. 3번은 이름이 FPGA_err로, 이 버전을 FPGA에 올렸다가 문제를 만난 것으로 짐작된다.
- 4번은 1번의 개선을 모두 되살린 최종본이다. 여기에 실물을 위한 수정 둘이 더해졌다. game_tick의 분주값이 833333에서 16000으로 바뀌었고, `ifdef APP_PROTO 아래에 clk_o 출력이 추가되었다.

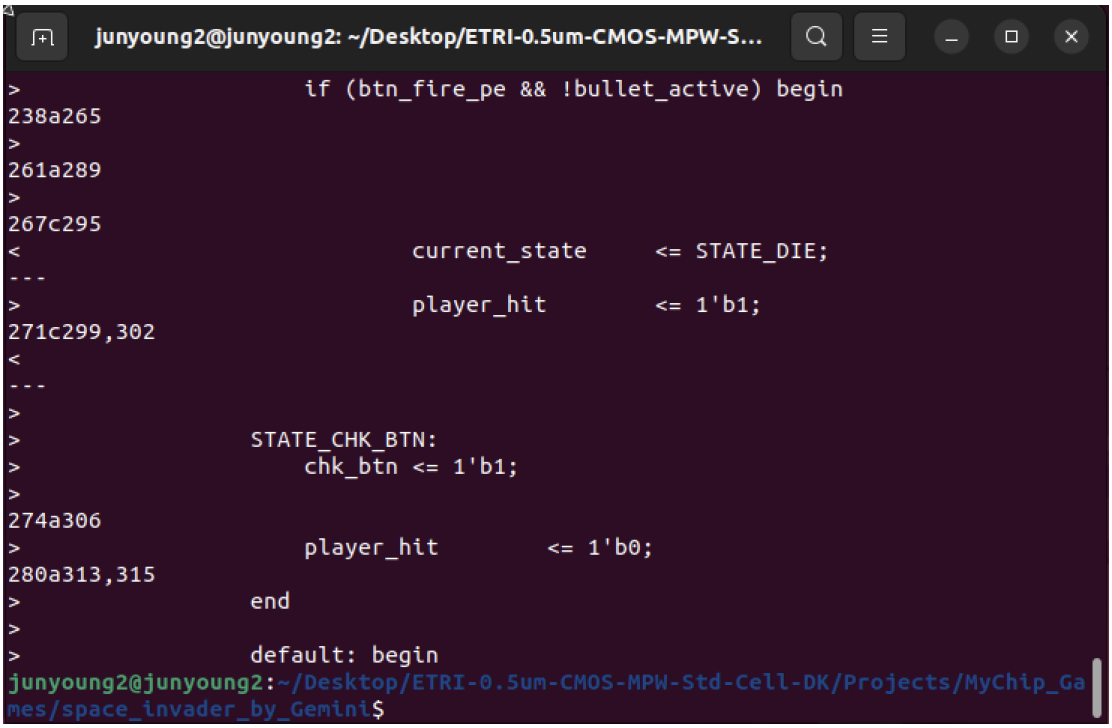



Fig. 2. 0번과 1번 RTL의 diff 일부. STATE_CHK_BTN의 추가와 player_hit 플래그의 도입이 보인다.

분주값의 변화가 특히 중요하다고 생각한다. 833333은 50MHz를 60으로 나눈 값이고, 16000은 1MHz를 60으로 나눈 값에 가깝다. 시뮬레이션에서 50MHz를 가정하고 만든 회로를 1MHz로 도는 검증 장비에 올리면 게임이 쉰 배 가까이 느려진다. 3번에서 만난 FPGA 오류의 성격이 이 계산으로 설명된다. 절대적인 클럭 수보다 내 회로가 어떤 클럭 위에서 도는지가 문제였다. 지난주 총평에서 속도의 숫자보다 논리를 보라고 하신 뜻을 이제 알 것 같다.

## 2.2 실행으로 확인

make의 실행 순서도 하나 배웠다. make를 인자 없이 실행하면 다섯 후보 중 하나를 골라 space_invaders_engine_glcd.v로 링크하는 단계가 먼저다. 그 링크 없이 make run부터 하면 모듈을 찾지 못한다는 오류가 난다(Fig. 3). 파일의 연결도 Makefile 안의 명령 연쇄로 구현되어 있다는 것을 오류를 통해 확인했다. 4번을 링크하고 실행한 화면이 Fig. 4다. 버튼을 누르는 만큼만 발사되고 포의 움직임도 프레임에 맞춰 차분해진 것을 확인했다.

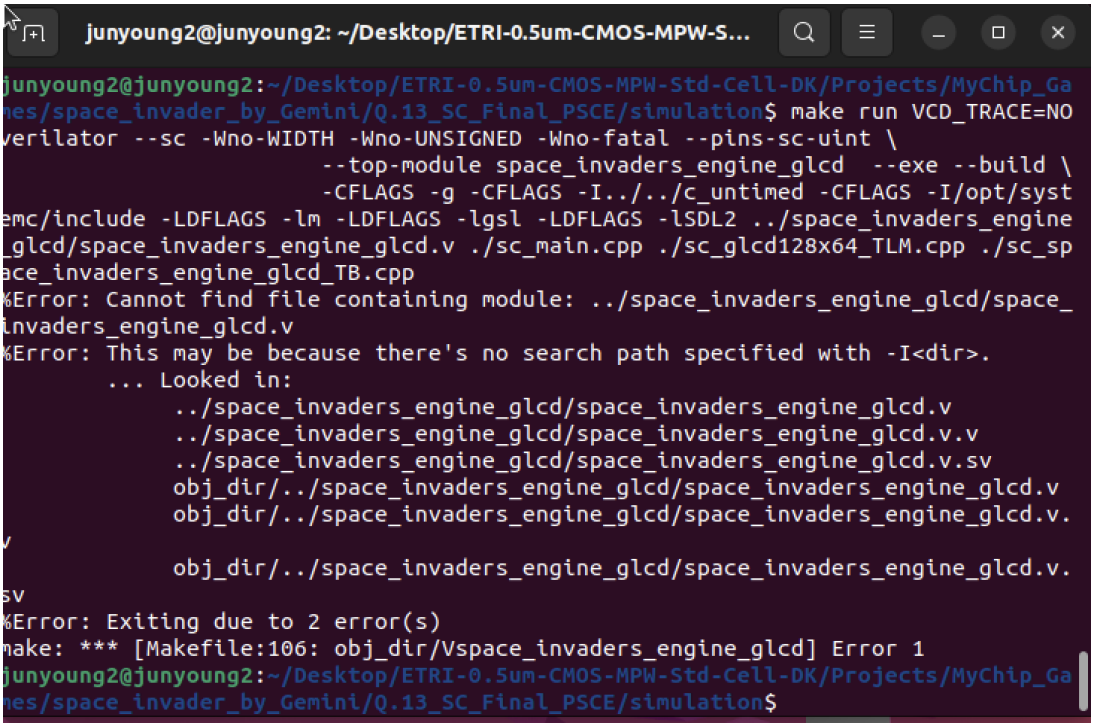



Fig. 3. RTL 링크 없이 make run을 실행했을 때의 오류. make가 먼저 RTL을 링크해야 함을 확인했다.

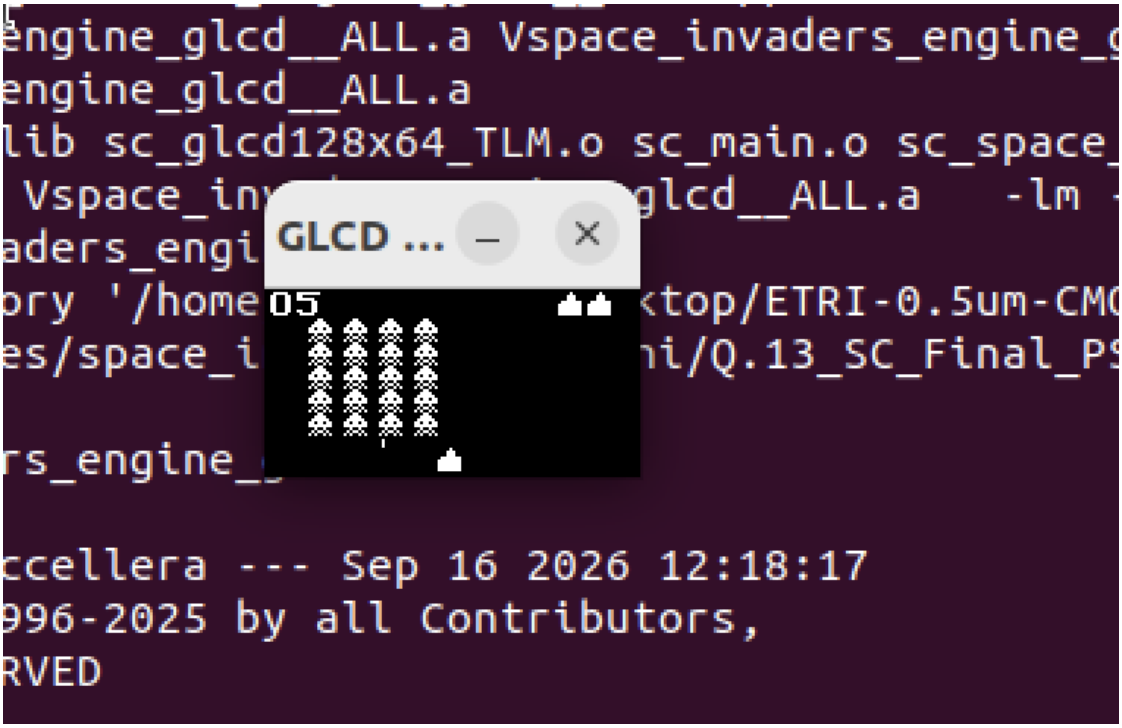



Fig. 4. Q.13에서 4번(Ok) RTL을 링크해 실행한 화면. 단발 사격과 프레임 단위 이동이 적용되어 있다.

### 2.3 Q.14의 다섯 버전: 대화에서 나온 코드들

Q.14의 후보는 Score, No_Score, v_1, v_2, v_3다(Fig. 5). Talks의 대화가 고스란히 파일로 남은 셈이다. No_Score가 인공지능에게 건넨 원본이고, v_1부터 v_3까지가 대화 끝에 나온 재설계다. 최종본인 v_3를 링크하고 빌드해 실행했더니 게임이 정상 동작했다(Fig. 6). 대화 속에서만 보던 코드가 시뮬레이션에서 도는 것을 확인했다.

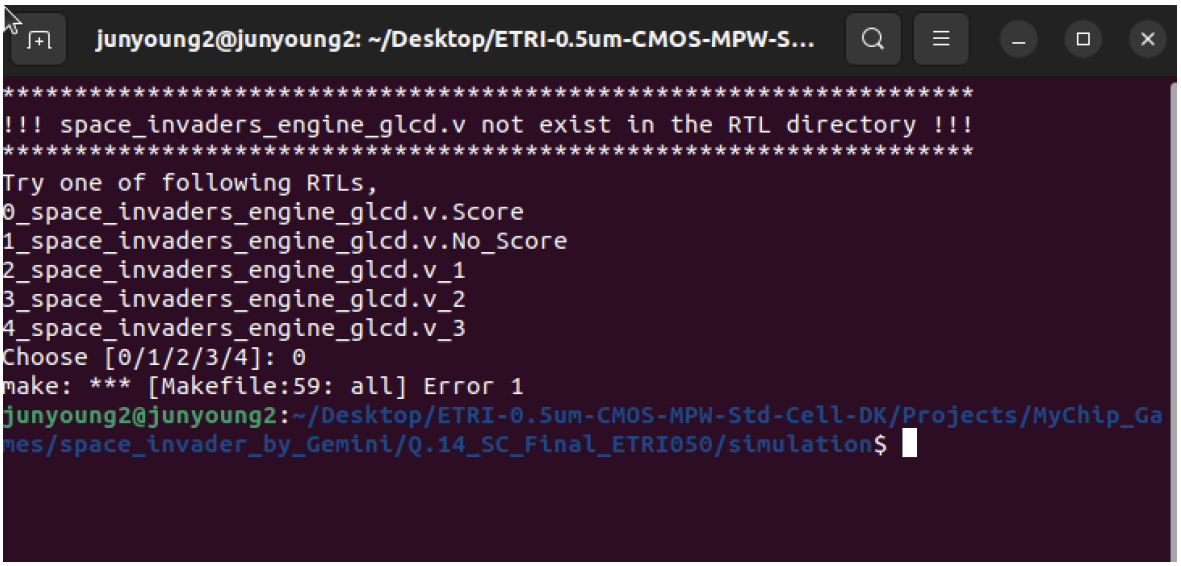



Fig. 5. Q.14 시뮬레이션의 RTL 선택 메뉴. Score와 No_Score, 그리고 대화 끝에 나온 v_1부터 v_3까지가 후보다.

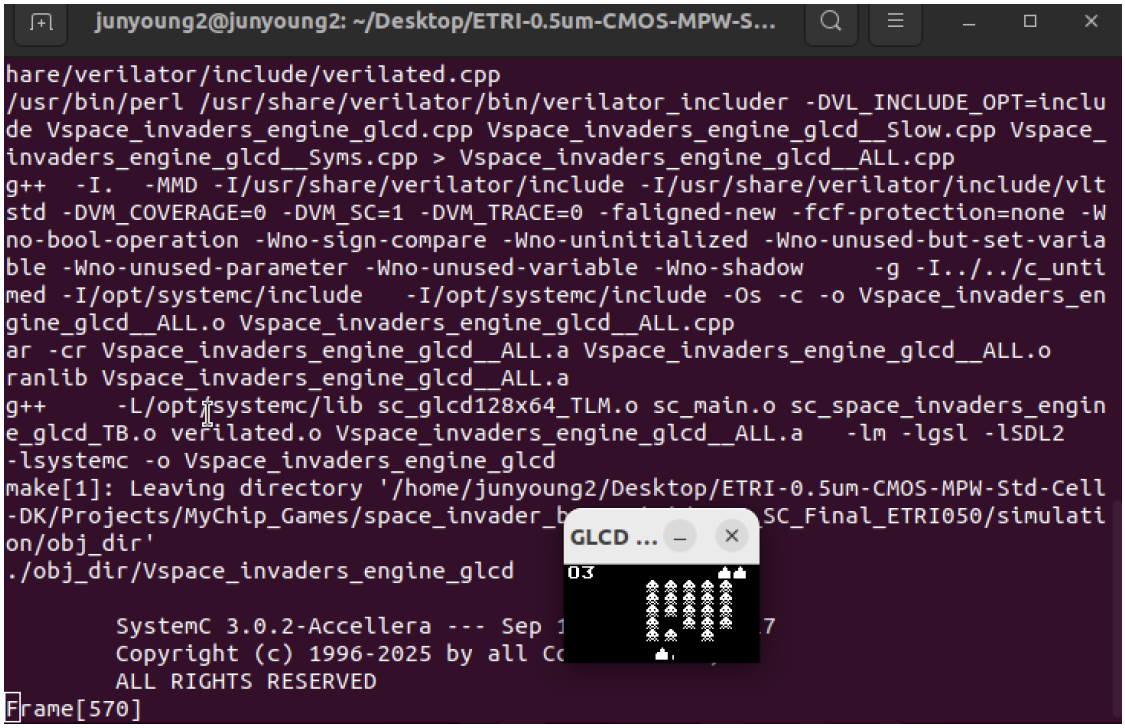



Fig. 6. Q.14에서 v_3를 링크하고 빌드에 성공해 실행한 화면.

## 3. Makefile이 부르는 도구들

이번에는 실행 결과가 아니라 Makefile 자체를 들여다보았다. simulation/Makefile의 기본 타깃은 배시 선택 스크립트로 RTL을 링크한다. run 타깃은 Verilator가 베릴로그를 SystemC C++로 변환하고(--sc), perl의 verilator_includer가 소스를 모으고, g++가 컴파일한 뒤 ar과 ranlib로 정적 라이브러리를 만들고, 마지막으로 SDL2와 SystemC, gsl과 링크해 실행 파일을 만든다. VCD_TRACE 변수로 파형 추적을 켜고 끈다.

환경 차이도 겪었다. 배포된 Makefile은 최신 Verilator를 가정하는데 내 가상 머신의 Verilator는 우분투 저장소의 구버전이라 -Wno-WIDTHTRUNC와 --timing 옵션을 모른다. --trace를 켜면 SystemC 3.0에서 바뀐 추적 인터페이스와도 충돌한다. 옵션을 정리하고 VCD_TRACE=NO로 빌드해 해결했다.

도구의 버전 차이가 Makefile의 가정과 어긋날 때 어느 층을 고쳐야 하는지 감이 생겼다.

다른 디렉토리의 Makefile도 같은 눈으로 읽었다. Q.14의 ETRI050/simulation에는 iverilog의 VPI를 쓰는 테스트벤치(vpi_stub.cpp)가 있어 합성된 넷리스트를 검증하는 자리임을 알 수 있다. application/fpga의 tcl 스크립트는 Quartus가 읽는 대본이고, application/firmware의 Makefile은 arduino-cli를 부른다. 같은 make라는 진입점 아래에서 단계마다 다른 도구가 바통을 넘겨받는 구조다.

## 4. 에뮬레이션과 검증 보드 준비

다음 주에 배포되는 검증 보드는 FPGA와 Raspberry Pi Pico를 결합한 장비다. 갱신된 저장소에는 이에 대비한 코드가 들어왔다. emulation 디렉토리에는 PSCE-MI와 PSCE-TRANS, 그리고 래퍼 베릴로그가 있고, application 디렉토리는 세 갈래로 나뉜다. firmware는 Pico에 올릴 스케치로 RP2040의 PWM 라이브러리를 포함한다. fpga는 Quartus 프로젝트를 만드는 tcl과 핀 배정 파일이다. run은 키보드 입력을 다루는 SystemC 코드다. Pico가 PWM으로 클럭을 만들어 FPGA에 공급하는 구조라면, 4번의 분주 수정이 1MHz 기준으로 바뀐 이유도 설명이 된다.

읽어 보라고 소개된 글 두 편도 이 맥락에서 읽었다. 2007년의 EE Times 기고문은 소프트웨어 비중이 커진 SoC는 첫 실리콘 전에 소프트웨어를 깨워야 하며, 그래서 SoC의 90퍼센트 이상이 FPGA 프로토타입을 거친다고 썼다. 올해의 vlsi.kr 글은 에뮬레이션과 프로토타이핑 팀이 pre-silicon 단계에서 운영체제와 드라이버를 먼저 깨우는 역할을 한다고 정리했다. 시뮬레이션을 거쳐 FPGA 검증 보드로 넘어가는 우리의 순서가 업계의 pre-silicon 검증 흐름의 축소판임을 알 수 있었다.

도구 준비 상황은 이렇다. arduino-cli는 가상 머신(ARM64 우분투)에 ARM64 판을 설치하고 RP2040 코어까지 붙였다. Quartus는 x86-64 전용이라 ARM 리눅스에 바로 설치되지 않는다. 별도의 x86 리눅스 환경을 마련해 보드 배포 전까지 FPGA 빌드를 성공시켜 둘 계획이다.

## 5. 마무리

지난주 노트에서 키보드 입력에 대해 움직임이 이산적으로 튄다고 썼는데, 총평에서 이 표현이 적절한지 물으셨다. 이번에 계보를 따라가며 원인을 확인하고 나니 답을 정리할 수 있다. 0번의 비정상적인 움직임은 프레임과 무관하게 매 클럭 위치가 갱신된 결과였다. 시간축이 연속적으로 흐르는 하드웨어에서 이 현상에 이산적이라는 말은 맞지 않았다. 오히려 chk_btn으로 프레임당 한 번만 갱신하게 고친 지금의 동작이 이산적이라는 이름에 어울린다.

마지막으로 이번 주에 남은 생각을 적는다. 인공지능은 스무다섯 벌의 회로를 한 벌로 줄이는 설계를 순식간에 해 냈지만, 정작 그 코드에서 선언이 빠진 것과 언더플로는 사람이 잡았다. 반대로 사람은 칩 면적의 규모에 대한 감을 인공지능에게서 얻었다. 내 칩을 실제로 찍어 내는 공정을 앞두고, 이 분업이 어디까지 믿을 수 있는지 계속 확인해 가고 싶다.